FlexQL: A Flexible SQL-like Database Driver

Name: Digvijay Shalikrao Patil
Roll No: 25CS60R23
Repository: https://github.com/DigvijayPatil12/flexql.git

Contents

1 System Architecture **3**

2 Data Storage & Memory Management **4**
2.1 In-Memory Representation 4
2.2 Serialization and Cold-Boot Recovery 4

3 Indexing Strategy & Primary Key Resolution **5**
3.1 Execution Routing 5
3.2 Maintenance Overhead and Trade-offs 5

4 Query Caching (LRU) **5**
4.1 Data Structure Design 5
4.2 Invalidation Strategy and ACID Compliance 6

5 Time-To-Live (TTL) & Garbage Collection **6**
5.1 Passive Read-Time Verification 6
5.2 Active Background Purging Daemon 6
5.3 Architectural Trade-offs 7

6 Concurrency & Thread Pool Design **7**
6.1 Connection Handling via Thread Pool 7
6.2 Reader-Writer Lock Semantics 7

7 Asynchronous Disk I/O & Backpressure **8**
7.1 The AsyncDiskWriter Pipeline 8
7.2 The Necessity of Backpressure 8

8 Performance Analysis & Scalability **9**
8.1 Batch Size Optimization 9
8.2 Vertical Scalability Limit 9

9 Compilation Instructions **10**

10 Execution Instructions **10**
10.1 Interactive Client & Multithreading Demonstration 11
10.2 Performance Benchmarks & Unit Tests 11

1 System Architecture

The FlexQL database engine is designed as a high-concurrency, low-latency relational database management system, operating on a client-server paradigm over standard TCP/IP sockets. The overarching architectural philosophy of FlexQL is to aggressively decouple synchronous computational tasks from asynchronous I/O bottlenecks. By maintaining primary data structures entirely in random-access memory (RAM) and deferring disk persistence to background daemons, the system approaches theoretical hardware limits for transaction throughput.

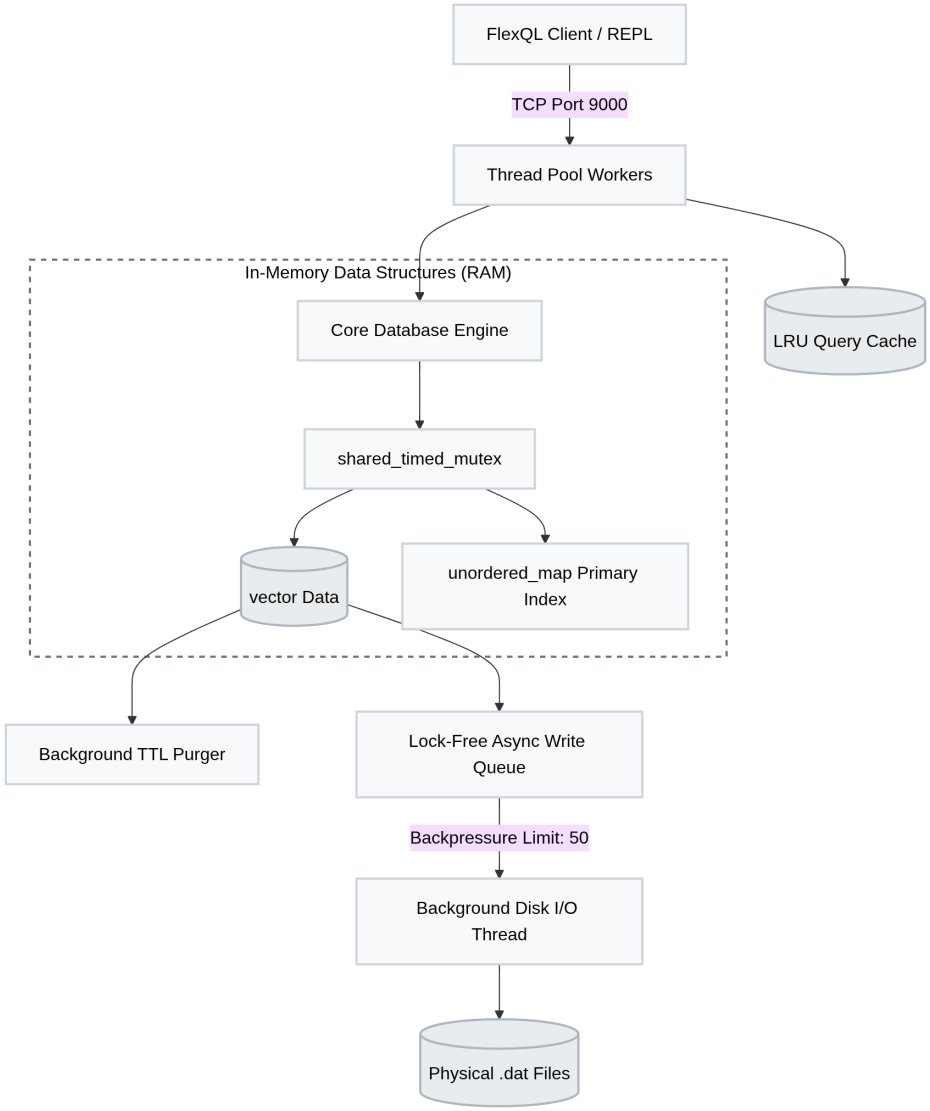

Figure 1: FlexQL Internal System Architecture and Threading Model

At the network boundary, the server initializes a POSIX socket listener bound to port 9000. Rather than adopting a naive, unbounded thread-per-connection model, FlexQL routes all incoming client socket descriptors into a unified, lock-free task queue. This queue is

consumed by a statically sized Thread Pool, dimensioned to match the underlying hardware's logical core count.

Once a worker thread picks up a connection, the client's raw SQL payload is passed to the Query Parser and Execution Engine. This internal engine acts as the central router, managing cache lookups, lock acquisitions, and asynchronous disk dispatches.

2 Data Storage & Memory Management

Memory layout dictates the ultimate ceiling of database performance. FlexQL employs a **Row-Major Storage** (N-ary storage model) architecture, optimizing for Online Transaction Processing (OLTP) workloads where the dominant access pattern involves fetching or appending entire tuples simultaneously.

2.1 In-Memory Representation

The core unit of data in FlexQL is the `Cell` structure. To support heterogeneous schema definitions without resorting to unsafe void pointers or heavy inheritance polymorphism, the `Cell` acts as a variant wrapper. It encapsulates a `std::string` for `VARCHAR` types, a `double` for `DECIMAL`, an `int` for standard integers, and a `time_t` object for `DATETIME` representations. An enumeration flag within the `Cell` tracks the active data type, ensuring strict runtime type safety during query evaluation.

These `Cell` objects are aggregated into a `Row` structure, which also contains a dedicated `time_t` field representing the tuple's Time-To-Live (TTL) expiration. The rows themselves are stored contiguously in a `std::vector<Row>` owned by a `Table` struct.

The choice of `std::vector` over a `std::list` for row storage is highly deliberate. By guaranteeing contiguous memory allocation, iterative operations such as full-table scans benefit immensely from modern CPU prefetching and cache locality. When the execution engine sequentially reads rows, entire blocks of tuples are loaded into the L1/L2 CPU cache, drastically reducing main memory access latency. The primary trade-off of this row-major design surfaces during column-specific aggregations; reading a single column across a million rows requires jumping across memory addresses, leading to inevitable cache misses.

2.2 Serialization and Cold-Boot Recovery

Data durability is achieved through flat-file serialization in the `data/` directory. Each table corresponds to a specific `.dat` file. To ensure atomic loads, the file format mandates a strict schema header defining column names and types, followed by pipe-delimited (`|`) tuple data. During a cold boot, the server engine dynamically scans the target directory. It reads the schema headers to reconstruct the in-memory `Table` objects, parses the delimited payloads, casts the strings back into their strongly-typed `Cell` formats, and fully repopulates the RAM vectors before accepting external TCP connections.

3 Indexing Strategy & Primary Key Resolution

To facilitate real-time query responses on datasets scaling up to 20 million rows, $O(N)$ sequential scans are computationally prohibitive for point-lookups. Consequently, FlexQL implements an aggressive, memory-resident **Primary Hash Index**.

Unlike traditional relational engines that default to B+ Trees to maintain sorted order for range queries, FlexQL prioritizes absolute retrieval speed for exact matches. The engine maintains a `std::unordered_map<int, size_t>` within every `Table` object. This structure maps the integer value of a Primary Key directly to its index address within the main `std::vector<Row>`.

3.1 Execution Routing

When the SQL parser encounters a `SELECT` statement, it performs a heuristic check on the `WHERE` clause. If the parser identifies an equality operator targeting the primary key (e.g., `WHERE ID = 1042`), the execution engine bypasses the iterative scan entirely. It performs a hash-lookup in the `unordered_map`. If the key exists, the engine receives the `size_t` index in $O(1)$ average time complexity, accesses the tuple directly via `table->rows[idx]`, and immediately returns the result.

3.2 Maintenance Overhead and Trade-offs

While the read performance of a Hash Index is mathematically optimal, it introduces specific write penalties. Every `INSERT` operation requires an additional allocation and hash computation to update the index map. Furthermore, because the map stores raw `size_t` vector indices, any operation that alters the fundamental memory layout of the `std::vector`—such as the background garbage collector removing expired rows—invalidates the entire index. To rectify this, FlexQL must perform a full index rebuild, iterating over the surviving rows and mapping their new indices. Lastly, the Hash Index provides zero utility for range queries (e.g., `WHERE BALANCE > 5000`), which are inherently forced to fall back to the $O(N)$ sequential scan path.

4 Query Caching (LRU)

To mitigate the CPU overhead of repetitive query parsing and execution, particularly for complex `INNER JOIN` operations, FlexQL implements an application-layer **Least Recently Used (LRU) Cache**.

4.1 Data Structure Design

The LRU Cache is engineered using a composite data structure to guarantee $O(1)$ time complexity for both data retrieval and cache eviction. It utilizes a `std::list<std::pair<std::string, std::string>>` acting as a doubly-linked list to maintain the chronological access order. The

front of the list represents the most recently accessed query, while the back represents the oldest.

Coupled with this list is a `std::unordered_map<std::string, std::list<...>::iterator>`. The map uses the raw SQL query string as the key, pointing directly to the memory node within the doubly-linked list. Upon a cache hit, the result payload is instantly returned to the network socket, and the node is spliced to the front of the list in $O(1)$ time, circumventing the need to acquire read locks on the core tables. If the cache reaches its predefined capacity, the tail of the linked list is popped, and its corresponding key is erased from the hash map.

4.2 Invalidation Strategy and ACID Compliance

Cache coherency is the primary challenge in distributed database design. To guarantee that a client never reads stale or logically deleted data, FlexQL employs a coarse-grained, aggressive invalidation strategy. The execution engine monitors the query type at the entry point. The moment any state-mutating command is issued—be it an `INSERT`, `CREATE TABLE`, or `DROP TABLE`—the entire LRU cache is systematically flushed.

While this ensures absolute data integrity, it represents a conscious trade-off. In a highly write-intensive environment, the cache hit ratio degrades rapidly, rendering the cache overhead detrimental to performance. However, for read-heavy analytical workloads, this mechanism preserves immense computational resources.

5 Time-To-Live (TTL) & Garbage Collection

Memory constraints dictate that an in-memory database cannot accumulate data indefinitely. FlexQL implements a hybrid Time-To-Live (TTL) architecture, utilizing both passive read-time filtering and active background garbage collection to ensure memory hygiene without sacrificing read accuracy.

5.1 Passive Read-Time Verification

The first layer of defense against stale data occurs directly within the execution engine. Every `Row` object inherently possesses an `expires_at` timestamp. When a client issues a `SELECT` or `JOIN` query, the engine iterates through the target tuples and evaluates this integer timestamp against the operating system's current epoch time (`std::time(nullptr)`). If the tuple has expired, the engine simply skips it during the projection phase. This "lazy evaluation" guarantees that a client will never receive logically deleted data, even if the physical memory has not yet been reclaimed.

5.2 Active Background Purging Daemon

While passive verification ensures logical correctness, physical RAM must eventually be freed to prevent Out-Of-Memory (OOM) faults. FlexQL achieves this via a detached daemon thread dedicated to garbage collection.

Upon server initialization, this thread enters an infinite sleep-wake cycle, typically awakening every 10 seconds. When active, it iterates through all registered `Table` objects. To ensure thread safety, the daemon must acquire a `std::unique_lock` on the table's `shared_timed_mutex`. Once exclusive access is granted, the daemon utilizes the Erase-Remove idiom (`std::remove_if` followed by `vector::erase`) to physically deallocate the expired `Row` objects from the heap.

5.3 Architectural Trade-offs

The active garbage collection phase introduces a significant, albeit necessary, performance penalty. Because erasing elements from the middle of a `std::vector` shifts the memory addresses of all subsequent elements, the Primary Hash Index is immediately invalidated. The daemon is forced to trigger a full index rebuild before releasing the write-lock. During this rebuilding phase, all read and write operations targeting that specific table are effectively stalled. This "stop-the-world" pause for individual tables is a classical trade-off in systems engineering: sacrificing transient tail-latency to guarantee long-term memory stability.

6 Concurrency & Thread Pool Design

FlexQL is engineered to sustain massive multi-client environments, managing thousands of concurrent socket connections. The threading model is explicitly designed to maximize CPU utilization while preventing thread starvation and context-switching cascades.

6.1 Connection Handling via Thread Pool

Operating systems enforce strict limits on file descriptors and thread stack allocations. A naive server implementation spawning a new `std::thread` for every incoming connection will quickly exhaust system resources and crash under benchmarking loads. To counteract this, FlexQL implements a **Fixed-Size Thread Pool**.

The pool is initialized with a predetermined number of worker threads (e.g., matching the host machine's `std::thread::hardware_concurrency()`). These workers sit idle, waiting on a `std::condition_variable`. When the primary server thread accepts a new socket connection via `accept()`, it does not process the payload. Instead, it pushes the socket descriptor into a thread-safe task queue and signals the condition variable. An idle worker awakens, dequeues the socket, processes the complete SQL transaction lifecycle, and returns to the pool.

6.2 Reader-Writer Lock Semantics

Within the execution engine, data integrity is maintained using `std::shared_timed_mutex`, an implementation of the Reader-Writer lock paradigm.

- **Readers:** `SELECT` queries acquire a `std::shared_lock`. This permits an infinite number of simultaneous readers to access the table concurrently, yielding immense read throughput.

- **Writers:** `INSERT`, `CREATE`, and Garbage Collection operations acquire a `std::unique_lock`. This ensures exclusive access, blocking all other readers and writers until the mutation is fully committed to RAM.

The primary vulnerability of this design is the potential for "Reader Starvation" or "Writer Starvation." In a scenario where clients are continuously issuing `SELECT` statements, the `shared_lock` may never fully release, causing pending `INSERT` statements to queue indefinitely. Future iterations of FlexQL could mitigate this by implementing lock-escalation or priority-queued mutexes.

7 Asynchronous Disk I/O & Backpressure

The most sophisticated optimization in the FlexQL architecture is the decoupling of the network interface from physical disk latency. Standard databases often block the client connection while flushing the transaction log to an SSD, capping throughput at the hardware's IOPS limit. FlexQL circumvents this entirely.

7.1 The AsyncDiskWriter Pipeline

When an `INSERT` query modifies the in-memory vector, the operation is considered logically complete. The raw, serialized string payload is immediately pushed into a concurrent queue managed by the `AsyncDiskWriter` class. The worker thread then instantly returns an "OK" acknowledgment to the TCP client, achieving sub-millisecond response times.

A separate, dedicated background thread continuously monitors this queue. When data is available, it acquires a lightweight lock, pops the payload, and performs an `std::ofstream` append operation to the corresponding `.dat` file.

7.2 The Necessity of Backpressure

While asynchronous I/O drastically increases throughput, it introduces a severe systemic risk. A modern CPU parsing `INSERT` batches can easily process 1,000,000 rows per second, drastically outpacing the write speed of a standard NVMe SSD. Without intervention, the `AsyncDiskWriter` queue will expand infinitely, consuming gigabytes of RAM until the Linux OOM-Killer forcefully terminates the server process.

To prevent this catastrophic failure, FlexQL enforces a strict **Backpressure Mechanism** utilizing a `std::condition_variable`. The queue is strictly bounded to a maximum limit of 50 batches.

```
// Producer Thread (Worker)
std::unique_lock<std::mutex> lock(queue_mutex);
cv_producer.wait(lock, [this]() { return write_queue.size() < 50; });
write_queue.push(batch_data);
cv_consumer.notify_one();
```

Listing 1: Backpressure Implementation Logic

If the network workers fill the queue to 50, the `cv_producer.wait()` condition forces the CPU to stall. The fast network threads are put to sleep, completely throttling the ingestion rate until the disk thread can flush the data and clear space in the queue. This trades raw, unbounded speed for absolute systemic stability.

8 Performance Analysis & Scalability

The system was benchmarked on standard hardware using a C++ benchmarking suite designed to test network overhead and raw vertical scalability limits.

8.1 Batch Size Optimization

The first experiment evaluated the impact of network packet overhead. Inserting rows with a batch size of 1 requires a full TCP round-trip, context switch, and SQL parsing phase for every individual tuple. The results confirmed that single-row inserts yield incredibly poor throughput. By increasing the batch size (grouping multiple tuples into a single `INSERT` statement), the network overhead was amortized. Throughput scaled linearly, ultimately reaching an optimal saturation point at a batch size of 5,000 rows.

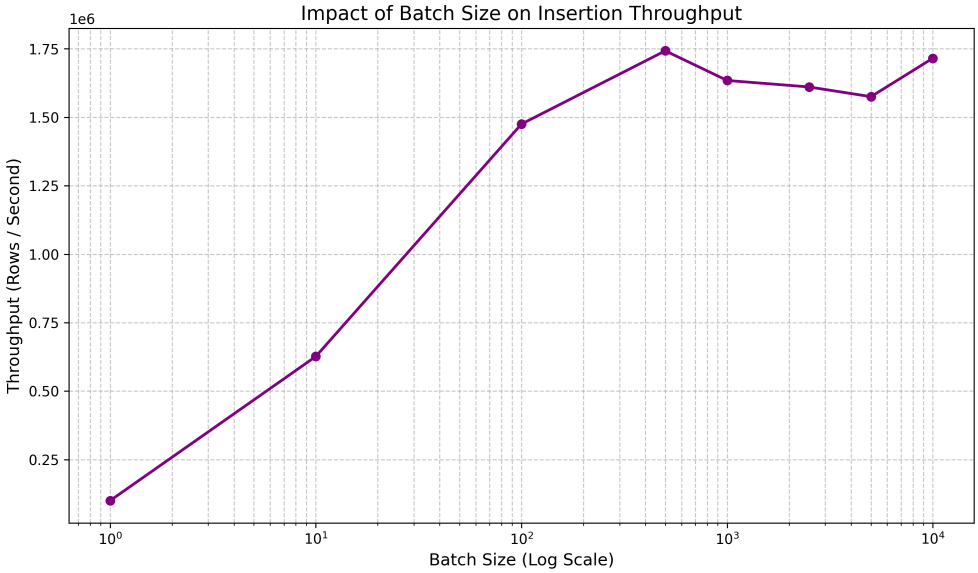

Figure 2: Impact of Batch Size on Insertion Throughput

8.2 Vertical Scalability Limit

Utilizing the optimized 5,000-row batch, the engine was subjected to a stress test scaling logarithmically up to 20,000,000 rows. The system successfully ingested the massive dataset without encountering memory exhaustion, verifying the effectiveness of the backpressure design. At approximately 15 million rows, the throughput curve began to flatten. This flattening indicates that software-level lock contention was no longer the limiting factor;

rather, the system had successfully pushed the bottleneck to the absolute physical write bandwidth of the underlying solid-state drive.

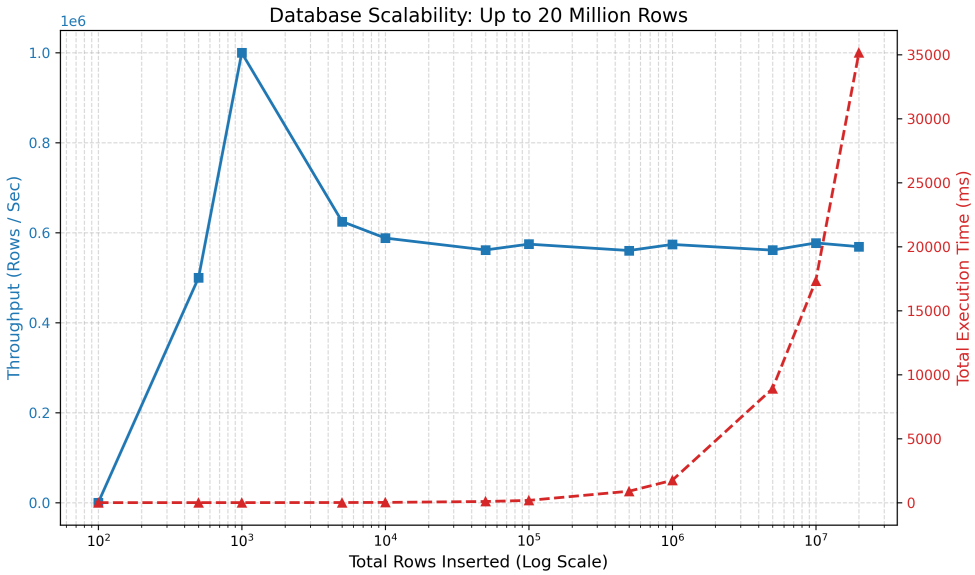

Figure 3: Database Scalability Limit (Up to 20 Million Rows)

9 Compilation Instructions

FlexQL requires a modern Linux environment (Ubuntu 20.04+) and a compiler supporting the C++14 standard. The `-O3` optimization flag is strictly required to ensure maximum execution speed.

```
# 1. Compile the Multithreaded Server Daemon
g++ -std=c++14 -pthread -O3 -I./include src/server/server.cpp -o bin/
    flexql-server

# 2. Compile the Interactive Client REPL
g++ -std=c++14 -I./include src/client/flexql_client.cpp src/client/main.
    cpp -o bin/flexql-client

# 3. Compile the Scalability Benchmark Suite
g++ -std=c++14 -O3 -I./include src/client/flexql_client.cpp tests/
    benchmark_flexql.cpp -o bin/run-benchmark
```

Listing 2: System Compilation Commands

10 Execution Instructions

Before executing any client or benchmark, the main database server must be initialized. It will bind to Port 9000 and wait for incoming TCP connections.

```
# Terminal 1: Initialize the Background Server Daemon
./bin/flexql-server
```

Listing 3: Start the Server

10.1 Interactive Client & Multithreading Demonstration

To interact with the database using standard SQL commands, launch the interactive client. To test the server's multithreaded connection handling (the Thread Pool), you can open multiple terminals and launch a client in each simultaneously.

```
# Terminal 2: Launch the primary interactive client
./bin/flexql-client

# Terminal 3 (Optional): Launch a second client to demonstrate concurrency
./bin/flexql-client
```

Listing 4: Launch Interactive Clients

Once connected, the terminal enters a Read-Eval-Print Loop (REPL). To safely disconnect from the server, use the `.exit` command.

```
flexql> CREATE TABLE STUDENT (ID INT, NAME VARCHAR);
flexql> INSERT INTO STUDENT VALUES (1, 'Alice');
flexql> SELECT * FROM STUDENT;
flexql> .exit
```

Listing 5: REPL Usage

10.2 Performance Benchmarks & Unit Tests

The system includes automated testing suites to verify structural integrity and vertical scalability. These scripts should be executed in a separate terminal while the server is actively running in Terminal 1.

```
# To run the massive dataset insertion benchmark (e.g., 10 Million Rows)
./bin/run-benchmark 10000000

# To execute the core data-structure unit tests
./bin/run-benchmark --unit-test
```

Listing 6: Run Benchmarks and Tests